

AGENSIBLE (TuAgendita)

Agenda Personalizada y Accesible para Estudiantes con Discapacidad Intelectual

CEE Fundación Purísima Concepción de Granada

ESTÁNDAR TÉCNICO OFICIAL Y STACK COMÚN

Revisión de Tareas, Directrices de Desarrollo y Homogeneización del Sprint 1

Asignaturas: Dirección y Gestión de Proyectos (DGP) &
Metodologías de Desarrollo Ágil (MDA)

Grado: Grado en Ingeniería Informática — Universidad de Granada

Subgrupo 2: *Los Gantt-sters*

Coordinador del Proyecto: Juan Luis Jiménez Urbano

Fecha: Noviembre 2026

Documento de uso interno para la coordinación técnica y estandarización del equipo de desarrollo.

Índice

1. Propósito y Justificación de la Estandarización **1**
1.1. Criterios de Idoneidad para el Cliente 1

2. Tabla Maestra del Stack Tecnológico Oficial **1**

3. Auditoría y Revisión de Tareas Realizadas (Sprint 1) **2**
3.1. HUT-01 / SCRUM-154: Infraestructura Cloud y Conexión Supabase 2
3.2. HU-02 / SCRUM-54: Inicio de Sesión Estándar para Supervisores (Golden Story) . . 2
3.3. DevOps y Configuración de Calidad 3

4. Checklist de Entrega Obligatoria antes de cada Commit **3**

1. Propósito y Justificación de la Estandarización

Durante las primeras sesiones de desarrollo convivieron en el repositorio dos enfoques técnicos: la configuración inicial con **Vite** (en las tareas técnicas de infraestructura BaaS) y la configuración con **Expo + React Native Web** (para el soporte táctil y la suite de calidad).

Para evitar inconsistencias, conflictos en la integración continua (*CI/CD*) y garantizar que los 6 miembros del equipo trabajen sobre el mismo entorno, se formaliza la siguiente decisión:

Decisión Oficial de Equipo: Adopción de Expo + React Native Web

Se adopta formalmente EXPO + REACT NATIVE WEB (`expo-router`) como stack único de la aplicación. Queda completamente descartado el uso de Vite.

1.1. Criterios de Idoneidad para el Cliente

- **Dispositivo objetivo prioritario:** El colegio utilizará la aplicación en **tablets en formato horizontal** por estudiantes con necesidades educativas especiales. React Native Web previene el zoom accidental, gestiona toques táctiles y optimiza el diseño en franjas.
- **Soporte multimedia nativo:** La aplicación requiere de avisos acústicos, síntesis de voz y renderizado de gráficos vectoriales (*Time Timer* y pictogramas ARASAAC), cubiertos por `expo-audio` y `react-native-svg`.
- **Despliegue dual:** Se compila limpiamente para despliegue web estático en **Vercel** (`npx expo export -p web`) y permite compilar una versión instalable (APK) sin cambiar código si el centro lo solicita.
- **Integración Continua validada:** Los flujos de GitHub Actions (`build-and-test.yml` y `ci.yml`) están configurados para Node 22 y Jest-Expo, pasando actualmente con 100 % de éxito.

2. Tabla Maestra del Stack Tecnológico Oficial

Todo desarrollo nuevo en el repositorio debe respetar estrictamente esta selección tecnológica:

Ámbito	Tecnología Oficial	Detalles y Versión
Runtime / Entorno	**Node.js v22 LTS**	Fijado en `.nvmrc` y en GitHub Actions
Gestor de paquetes	**npm**	Mantener coherente `package-lock.json`
Lenguaje	**TypeScript (6.0)**	Compilación estricta (`strict: true`)
Frontend Base	**Expo Web (58.0)**	Con `react-native-web ^0.21` y React 19
Enrutamiento	**Expo Router (58.0)**	Enrutador basado en carpetas (`app/`)
Estilos e Interfaz	**StyleSheet React Native**	Contrastes accesibles WCAG 2.1 AA
Backend & BaaS	**Supabase (PostgreSQL)**	Base de datos con RLS y Auth con JWT
Comunicación Aumentativa	**API ARASAAC**	Catálogo oficial de pictogramas
Testing Unitario	**Jest + Jest-Expo**	Con `@testing-library/react-native`
Calidad & Accesibilidad	**ESLint 9**	Con `eslint-plugin-jsx-a11y`
Formateo	**Prettier**	Ejecución con `format:check`
CI / CD	**GitHub Actions**	Workflows obligatorios en cada Pull Request
Despliegue Cloud	**Vercel**	Conectado a la rama `main`

Tabla 1: Matriz de tecnologías oficiales y estandarizadas.

3. Auditoría y Revisión de Tareas Realizadas (Sprint 1)

Se ha realizado una auditoría completa del código implementado hasta la fecha para asegurar su total alineación con el estándar Expo y los criterios de aceptación en Jira:

3.1. HUT-01 / SCRUM-154: Infraestructura Cloud y Conexión Supabase

- **SCRUM-155 (T-T01.1):** Despliegue continuo en Vercel. Se adaptó la configuración para permitir el empaquetado de Expo Web sin dependencias residuales de Vite.
- **SCRUM-157 (T-T01.3):** Conexión de cliente y variables de entorno. Se implementó retrocompatibilidad en `src/lib/supabaseClient.ts` para aceptar tanto variables con prefijo `EXPO_PUBLIC_` como las anteriores `VITE_`.

3.2. HU-02 / SCRUM-54: Inicio de Sesión Estándar para Supervisores (Golden Story)

- **SCRUM-56 (T-02.2 Backend):**
  - Servicio de autenticación JWT y contraseñas cifradas en Supabase Auth (`RNF1.3`).
  - Control de roles RBAC (`tutor_docente`, `familiar`, `admin`, `especialista`) con redirección dinámica (`RF1.4`).
  - Política de seguridad contra enumeración de usuarios (`RF1.8`): mensaje genérico *Çredenciales incorrectas o usuario no autorizado."*
  - 11 tests unitarios automáticos cubriendo el 100 % de las ramas lógicas.
- **SCRUM-55 (T-02.1 Frontend):**
  - Pantalla de login accesible (`app/login.tsx`) desarrollada íntegramente con componentes de React Native Web.
  - Cumplimiento WCAG 2.1 AA con etiquetas accesibles, roles semánticos y mensajes de alerta para lectores de pantalla.

  - Validación reactiva de formulario y selector Demo para agilizar las pruebas de QA.
  - 6 tests de integración con `userEvent` simulando interacción real.
- **SCRUM-57 (T-02.3 QA):** Asignada al Gestor de Calidad (Javier Miñán) para pruebas avanzadas de expiración de sesión y estrés de tokens JWT.

3.3. DevOps y Configuración de Calidad

- Se homogeneizó el archivo `tsconfig.json` incluyendo `"types": ["jest"]` para que el chequeo de tipos de TypeScript (`tsc --noEmit`) sea impecable.
- Se actualizó `.prettierignore` para excluir artefactos de documentación y no entorpecer la validación del linter.

4. Checklist de Entrega Obligatoria antes de cada Commit

Para evitar que se suban Pull Requests con errores que bloqueen el pipeline de GitHub Actions, cada desarrollador **debe ejecutar localmente los siguientes 4 comandos**:

Protocolo de Verificación Local (4 Checks Verdes)

```
1. npm test -- --ci              (Tests unitarios con Jest)
2. npx tsc --noEmit               (Chequeo estricto TypeScript)
3. npm run lint                 (Calidad y accesibilidad ESLint)
4. npm run format:check         (Formateo de código Prettier)
```

Nota: Si el comando de Prettier detecta desalineaciones, ejecutar simplemente `npm run format` para que corrija los archivos automáticamente antes de confirmar el commit.

Los Gantt-sters — Subgrupo 2 · UGR 2026/2027
Çalidad, accesibilidad y rigor en el desarrollo ágil."